

Practice 10 详解答案

A. 单项选择题

1. **B**。引用参数可以直接访问并修改调用者传入的原始变量。
2. **C**。& 表示引用。
3. **C**。string note 是按值传递，函数中修改的是副本。
4. **C**。string& note 是对原始字符串的引用。
5. **A**。参数让函数之间的数据传递方式明确且可预测。
6. **C**。创建笔记需要把用户输入写回调用者的变量。
7. **B**。引用是已有变量的别名。
8. **A**。按值传递会复制对象；对象较大或调用频繁时开销更明显。
9. **B**。各函数通过参数接收、读取或更新同一份笔记数据。
10. **B**。第 9 课重在组织代码，第 10 课进一步关注数据如何在函数间流动。

B. 填空题

1. 值（按值）
2. 引用（按引用）
3. &
4. 数据
5. 复制（拷贝）

C. 判断题

1. **对**。形参是实参的一份独立副本。
2. **对**。引用形参是原始变量的别名。
3. **错**。按值传递时只会修改副本。
4. **对**。引用可以让多个函数读取或修改同一个对象。
5. **对**。string& note 是非常量左值引用参数。
6. **错**。只读且对象较大时通常使用 const string&；小型值或需要副本时可按值传递。
7. **对**。数据流就是数据在变量、函数和文件之间的传递过程。
8. **对**。复制一个字符串通常需要额外内存。
9. **对**。普通引用必须在声明时绑定到已有对象。
10. **对**。函数划分职责，数据流把这些职责连接起来。

D. 代码阅读

1. 按值传递

题中 `intx` 应写为 `int x`。修正后：

```
void change(int x)
{
    x = 100;
}

int main()
{
    int value = 10;
    change(value);
    cout << value;
}
```

输出是 **10**。 `x` 是 `value` 的副本，赋值不会影响 `value`。

2. 按引用传递

题中 `intmain` 应写为 `int main`。修正后，输出是 **100**：

```
void change(int& x)
{
    x = 100;
}
```

`x` 引用 `value` 本身，因此修改 `x` 就是修改 `value`。

3. note 的最终内容

最终值为：

```
Learn Functions #Programming
```

`addTag` 的参数是引用，因此 `+=` 会直接修改 `main` 中的字符串。

4. 设计问题

`note` 是 `createNote()` 的局部变量，函数结束后就销毁，其他函数无法查看或保存读入的笔记。应让调用者提供存储位置：

```
void createNote(string& note)
{
    getline(cin, note);
}
```

5. 合适的参数类型

```
void viewNote(const string& note)
{
    cout << note;
}
```

它不复制字符串，并且 `const` 保证查看函数不会修改笔记。

E. 简答题

1. 按值传递与按引用传递的区别

按值传递会创建实参的副本，函数操作副本，因此不会改变原始变量；它适合小型数据或函数确实需要独立副本的场景。按引用传递让形参成为原始变量的别名，函数可直接修改原始变量，也避免复制大型对象。只读场景通常使用 `const T&`，以兼顾效率和防止误修改。

2. 数据流为什么重要

多函数程序中，一个函数产生的数据往往是另一个函数的输入。例如，`createNote` 读取笔记、`addTag` 更新笔记、`saveNote` 写入文件、`viewNote` 显示笔记。明确的数据流让每个函数的输入、输出和职责清晰，便于测试、调试和扩展。

3. 何时优先使用按引用传递

当函数必须修改调用者的变量时，使用非常量引用，例如 `string& note`。当函数只读取较大的对象且不应修改它时，使用常量引用，例如 `const string& note`。对于 `int` 等小型数据，按值传递通常更简单。

F. 编程题

1. 更新用户名

```
#include <iostream>
#include <string>
using namespace std;

void updateUsername(string& username)
{
    username = "GuestUser";
}

int main()
{
    string username = "Alice";
    updateUsername(username);
    cout << username << '\n';
}
```

输出： GuestUser 。

2. 笔记编辑器

```
void appendReminder(string& note)
{
    note += " [Review Tomorrow]";
}

int main()
{
    string note = "Study Functions";
    appendReminder(note);
    cout << note << '\n';
}
```

输出： Study Functions [Review Tomorrow] 。

3. 不修改地查看

```
void viewNote(const string& note)
{
    cout << note << '\n';
}
```

const 使 note 在函数内只读，编译器会阻止 note += "..." 之类的误修改；& 则避免复制字符串。

4. 多函数数据流

```
#include <iostream>
#include <string>
using namespace std;

void createNote(string& note)
{
    cout << "Enter note: ";
    getline(cin, note);
}

void addTag(string& note)
{
    note += " #Programming";
}

void displayNote(const string& note)
{
    cout << "Final note: " << note << '\n';
}

int main()
{
    string note;
    createNote(note);
    addTag(note);
    displayNote(note);
}
```

main 拥有 note；创建和加标签函数可修改它，显示函数只读取它。

5. CLI Notes v6 基础

```
#include <fstream>
#include <iostream>
#include <limits>
#include <string>

using namespace std;

void createNote(string& note)
{
    cout << "Enter note content: ";
    getline(cin, note);
}

void viewNote(const string& note)
{
    if (note.empty()) {
        cout << "No note available.\n";
    } else {
        cout << "Current note: " << note << '\n';
    }
}

void saveNote(const string& note)
{
    ofstream file("notes.txt");
    if (!file) {
        cout << "Could not open notes.txt.\n";
        return;
    }
    file << note << '\n';
    cout << "Note saved.\n";
}

void loadNote(string& note)
{
    ifstream file("notes.txt");
    if (!file) {
        cout << "No saved note was found.\n";
        return;
    }
    getline(file, note);
    cout << "Note loaded.\n";
}
```

```

void searchNote(const string& note)
{
    string keyword;
    cout << "Enter keyword: ";
    getline(cin, keyword);

    if (!note.empty() && note.find(keyword) != string::npos) {
        cout << "Keyword found.\n";
    } else {
        cout << "Keyword not found.\n";
    }
}

int main()
{
    string note;
    int choice = 0;

    while (choice != 6) {
        cout << "\n1. Create\n2. View\n3. Save\n4. Load\n5. Search\n6. Exit\n";
        cout << "Choose: ";

        if (!(cin >> choice)) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Please enter a number.\n";
            continue;
        }
        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        switch (choice) {
            case 1: createNote(note); break;
            case 2: viewNote(note); break;
            case 3: saveNote(note); break;
            case 4: loadNote(note); break;
            case 5: searchNote(note); break;
            case 6: cout << "Goodbye.\n"; break;
            default: cout << "Invalid choice.\n";
        }
    }
}

```

程序没有全局变量。main 创建并拥有 note，再将它明确地传给各操作函数，这就是清晰的数据流。

G. 设计与思考挑战

设计 B 更好，因为依赖关系直接写在函数参数中：`createNote` 明确需要一个可修改的字符串，`viewNote` 明确只读取字符串。函数不依赖隐藏的全局状态，因此更容易单独测试、复用，也更容易同时管理多条笔记。`const string&` 还避免复制并防止查看操作意外改写数据。